

TME 3

U.E. ASTRE

La machine à café

On considère une machine à café avec les spécifications suivantes :

- initialement, la machine est en veille dans un état `idle`;
- à l'appui sur un bouton demandant un café (action `press`), la machine passe dans un état `add_sugar` où l'utilisateur peut ajouter du sucre;
- l'utilisateur peut ajouter autant de sucres qu'il le souhaite pendant `p2` unités de temps, en respectant un délai de `p1` unités de temps entre chaque appui sur le bouton;
- une fois le délai est dépassé, la machine délivre une tasse (`cup`) et passe dans un état `preparing` où elle prépare le café;
- au bout de `p3` unités de temps après la demande d'un café (donc un temps de préparation de `p3 – p2` unités de temps), la machine termine la préparation du café et le sert (`coffee`);
- dix unités de temps après avoir délivré un café et si aucune autre demande n'est faite, la machine retourne en veille (`idle`, `sleep`).

Modélisation

Question 1 : Rédiger un modèle `cafe.imi` au format IMITATOR qui correspond à la spécification donnée de la machine à café.

La syntaxe de votre automate devra être cohérente avec celle vue en cours pour les automates temporisés : une contrainte (dans une garde ou un invariant) est une conjonction de comparaisons de la forme $x \sim p$ ou $x \sim c$.

Question 2 : Existe-t-il des valeurs des paramètres `p1`, `p2` et `p3` pour lesquelles la machine à café ne peut jamais servir de café ? Justifier votre réponse à l'aide d'IMITATOR.

Analyse d'une instance

Question 3 : On considère l'instance des paramètres suivante : `p1 = 2`, `p2 = 5`, `p3 = 8`

Nous allons étudier le comportement de la machine à café avec ces paramètres en utilisant IMITATOR. Pour cela :

- instancier les paramètres dans le modèle : dans l'en-tête du modèle, remplacer `p1` par `p1=2`, `p2` par `p2=5`, et `p3` par `p3=8`
- générer le graphe des états symboliques.

Pour ces paramètres, combien de sucres peut-on commander au minimum ? Au maximum ?

Question 4 : Grâce à IMITATOR, nous allons synthétiser d'autres valeurs des paramètres pour lesquelles le comportement du système est le même que pour les valeurs ci-dessus. Effectuer une synthèse de paramètres par méthode inverse : on utilise l'opérateur `IM` pour indiquer la propriété (`property := #synth IM( ... )`)

Quelle est la contrainte résultat ? Que garantit-elle ? Comment peut-on le vérifier ?

Graphe des états symboliques

Question 5 : Générer le graphe des états symboliques de cet automate temporisé paramétré (`imitator cafe.imi -mode statespace [-draw-statespace full]`). Que remarque-t-on ?

Question 6 : L'optimisation algorithmique consistant à tester l'inclusion d'un état dans un autre, au lieu de l'égalité, s'utilise avec l'option `-comparison inclusion`. Relancer la génération du graphe des états symboliques avec cette option. Que constate-t-on quant à la terminaison ? Tester et discuter la génération avec différentes versions de l'option `comparison`.

Question 7 : À partir de l'espace d'états obtenu à la question précédente, déduire une contrainte nécessaire et suffisante permettant l'obtention d'un café avec du sucre. Expliquer la méthode utilisée.

Question 8 : Formaliser cette propriété dans la syntaxe d'IMITATOR.

Question 9 : Déterminer, à l'aide d'IMITATOR, les valeurs de paramètres pour lesquelles l'obtention d'un café sucré est possible. Ce résultat est-il cohérent avec votre résultat calculé manuellement plus haut ?

Question 10 : On souhaite synthétiser les valeurs de paramètres pour lesquelles il est impossible de commander un café avec 3 sucres ou plus. Définir un observateur qui capture ce comportement, puis effectuer une synthèse de paramètres.

Conception d'un système : le passage à niveau (le retour)

On considère un système de contrôle de passage à niveau.

Un capteur est installé à une certaine distance en amont du passage à niveau; le temps entre le moment où le train active le capteur (action `approche`) et celui où il arrive au passage à niveau (action `passe`) est d_{arrive} . Une fois que le train active le capteur, cela va enclencher la baisse de la barrière (action `baisse`); le temps entre le moment où le train active le capteur et celui où la barrière commence à descendre est $d_{\text{activeBarrière}}$. Le temps entre le moment où la barrière commence à descendre et celui où la barrière est entièrement baissée (action `enBas`) est d_{descend} .

Si le train passe quand la barrière est déjà baissée, le train s'éloigne instantanément, et la barrière met 5 unités de temps à remonter, puis le système repart dans son état initial. On suppose que des voitures, piétons ou vélos peuvent toujours passer tant que la barrière n'est pas complètement baissée. Si le train passe alors que la barrière n'est pas complètement baissée, une collision se produit, et le système part dans un état `crash`. Par souci de simplification, on suppose que le train suivant ne peut pas passer pendant que la barrière remonte.

Question 1 : Rédiger un (unique) automate temporisé paramétré au format IMITATOR dans un fichier appelé `train.imi` et qui modélise ce système.

On considère qu'il n'y a qu'une seule voie, que les trains ne passent que dans un seul sens et que, même si plusieurs trains peuvent se suivre, un second train n'arrive qu'une fois que le premier a quitté le passage à niveau et que la barrière est remontée.

La syntaxe de votre automate devra être cohérente avec celle vue en cours pour les automates temporisés : une contrainte (dans une garde ou un invariant) est une conjonction de comparaisons de la forme $x \sim p$ ou $x \sim c$.

Question 2 : Quelle est la propriété de sûreté de ce passage à niveau ? L'exprimer en CTL.

Question 3 : En utilisant IMITATOR, déterminer automatiquement toutes les valeurs des paramètres qui garantissent l'absence de collision.

Question 4 : On suppose désormais que, pendant que la barrière remonte, un nouveau train peut commencer à approcher. Modifier votre modèle pour prendre en compte ce cas.

Question 5 : Améliorer votre modèle de passage à niveau pour que les 3 temps considérés soient désormais des intervalles. Par exemple, le temps entre le moment où le train active le capteur et celui où il arrivera au passage à niveau devient un intervalle $[d_{\text{arrive}}^-, d_{\text{arrive}}^+]$.

Question 6 : Calculer l'ensemble des valeurs des paramètres pour lesquelles aucune collision ne peut se produire.

Question 7 : Améliorer votre modèle pour qu'un second train puisse arriver alors que le premier n'était pas encore sorti du passage à niveau. Peut-il y avoir en plus une collision entre les deux trains ? Dans quelle situation ?

Question 8 : Même question pour 3 trains. Que remarque-t-on sur la complexité du modèle ? Peut-on facilement généraliser à n trains ?

Annexes

Quelques options utiles pour utiliser IMITATOR (il est possible d'afficher toutes les options disponibles avec `-help`) :

- `-mode statespace -draw-statespace full` : générer et dessiner l'espace d'états
- `-depth-limit n` : limiter la profondeur de l'espace d'états à n (environ)
- `-comparison x` : comparer les nouveaux états générés avec la stratégie x (x peut être `none`, `equality`, `inclusion`, `including` ou `doubleinclusion`)